

Keras API cung cấp sẵn hàng chục hàm losses, metrics, layers, optimizers chuẩn (ví dụ: `loss="mse"`, `activation="relu"`). Thực tế phức tạp hơn:

1. Bài toán đặc thù yêu cầu hàm tính sai số hoàn toàn mới (VD: Phạt lỗi sai số dương nặng gấp đôi sai số âm).

2. Nghiên cứu tạo ra một **cấu trúc** Tầng Nơ-ron (Layer) chưa từng có.

3. Xây dựng **mạng đa luồng:** 3 nguồn dữ liệu vào (Multi-input) và 2 luồng kết quả xuất ra (Multi-output).

TensorFlow 2 cho phép dùng Python viết mọi thứ, đóng gói lại và dùng như Module nguyên bản.

Gemini Notebook

Tùy chỉnh Hàm mất mát (Custom Loss) - Huber Loss (1/2)



Ví dụ: Bài toán dự đoán giá cả. Lỗi (error = label - prediction).

MSE (Mean Squared Error): Phạt bình phương lỗi -> rất nhạy cảm nhiễu.
MAE (Mean Absolute Error): Trừng phạt tuyến tính -> không mượt quanh điểm 0.
Huber Loss: Hành vi giống MSE khi lỗi nhỏ, giống MAE khi lỗi lớn.

```python
def huber_fn(y_true, y_pred):
    error = y_true - y_pred
    is_small_error = tf.abs(error) < 1
    squared_loss = tf.square(error) / 2
    linear_loss = tf.abs(error) - 0.5
    # Dung tf.where() giong nhu np.where()
    return tf.where(is_small_error , squared_loss , linear_loss)
```

Gemini Notebook

Tùy chỉnh Hàm mất mát (Custom Loss) - Sử dụng (2/2)

```
# Truyen truc tiep ten ham huber vao phuong thuc compile
model.compile(loss=huber_fn , optimizer="nadam", metrics=["mae"])

# Huan luyen nhu binh thuong
model.fit(X_train, y_train, epochs=2)
```



Keras sẽ truyền y_true và y_pred (Tensor) vào huber_fn tự động trong mỗi batch. Tính sai số -> tự động truy vết lại để tính đạo hàm cập nhật ngược (backpropagation).

Gemini Notebook

Lưu ý khi tải (load) mô hình chứa Custom Loss



Vấn đề

Cấu trúc Keras không lưu trữ mã nguồn Python của hàm huber_fn. Khi dùng `keras.models.load_model()` ở script khác, Keras sẽ báo lỗi **"Unknown loss function"**.

Giải pháp

```
# Luu nhu binh thuong
model.save("my_model_with_a_custom_loss.h5")

# Khi load, ban BAT BUOC phai truyen tu dien (dictionary)
# khai bao mapping tu chuoi sang doi tuong Python
model = keras.models.load_model(
    "my_model_with_a_custom_loss.h5",
    custom_objects={"huber_fn": huber_fn}
)
```

Gemini Notebook

Xây dựng Custom Loss bằng OOP (Subclassing) (1/2)



Nếu hàm Loss yêu cầu siêu tham số (như **threshold=1.0** không cố định), sử dụng Hàm Python rất bất tiện. Giải pháp OOP: Kế thừa lớp **keras.losses.Loss**. Khai báo **get_config()** để tự động lưu tham số.

```
class HuberLoss(keras.losses.Loss):
    def __init__(self, threshold=1.0, **kwargs):
        self.threshold = threshold
        super().__init__(**kwargs)

    def call(self, y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < self.threshold
        squared_loss = tf.square(error) / 2
        linear_loss = self.threshold * tf.abs(error) - self.threshold**2 / 2
        return tf.where(is_small_error , squared_loss , linear_loss)
```

Gemini Notebook

Xây dựng Custom Loss bằng OOP (Subclassing) (2/2)



Cung cấp `get_config()` để lưu trữ tham số mạnh mẽ hơn.

```
    def get_config(self):
        base_config = super().get_config()
        # Cap nhat thong so cua minh kem theo thong so cha
        return {**base_config , "threshold": self.threshold}

# Chi can tao doi tuong thay vi day string
model.compile(loss=HuberLoss(2.), optimizer="nadam")

# Keras goi get_config de luu threshold=2. vao JSON Model.
model.save("my_custom_model_v2.h5")

# Khi load, custom_objects van la bat buoc!
model = keras.models.load_model("my_custom_model_v2.h5",
    custom_objects={"HuberLoss": HuberLoss})
```

Gemini Notebook

Tùy chỉnh Hàm Đánh giá (Custom Metrics)



Bạn có thể tái sử dụng Custom Loss cho Metric. Khác biệt: Loss dùng để huấn luyện, Metric in ra để đánh giá.

```
model.compile(loss="mse", optimizer="nadam", metrics=[huber_fn])
```



Vấn đề:
Metrics tiêu chuẩn Keras là 'Streaming metrics', tính luỹ kế qua các batch rồi chia trung bình. Hàm Python đơn giản sẽ bị tính sai luỹ kế.

Giải pháp nâng cao:
Kế thừa `keras.metrics.Metric`, quản lý các biến trạng thái bằng `self.add_weight()` và ghi đè hàm `update_state()`, `result()`.

Gemini Notebook

Tùy chỉnh Lớp ẩn không trạng thái (Stateless Custom Layers)



Khái niệm cốt lõi (Core Concept)

Sử dụng khi cấu trúc phi tuyến đặc biệt không chứa tham số cấu hình cần học (Weights/Biases). Keras cung cấp keras.layers.Lambda.

Ứng dụng: Đóng vai trò làm activation function hoặc preprocessing.

```
# Tao tang mu (exponential layer)
exponential_layer = keras.layers.Lambda(lambda x: tf.exp(x))

# Tích hợp vao Sequential Model
model = keras.models.Sequential([
    keras.layers.Dense(30, activation="relu"),
    keras.layers.Dense(1),
    exponential_layer
])
```

Gemini Notebook

Tùy chỉnh Lớp ẩn có trạng thái (Stateful Custom Layer) (1/2)



Khái niệm cốt lõi (Core Concept)

Để tạo Layer riêng có khả năng 'Học' hệ số W, b, kế thừa keras.layers.Layer. Gồm 3 hàm cốt lõi: __init__, build() (khởi tạo Weights khi nhận dạng Shape) và call() (thực thi Feed-Forward).

```
class MyDense(keras.layers.Layer):
    def __init__(self, units, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.activation = keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.kernel = self.add_weight(
            name="kernel", shape=[batch_input_shape[-1], self.units],
            initializer="glorot_normal")
```

Gemini Notebook

Tùy chỉnh Lớp ẩn có trạng thái (Stateful Custom Layer) (2/2)



```
    # Tiep tuc ham build
    self.bias = self.add_weight(
        name="bias", shape=[self.units], initializer="zeros")
    super().build(batch_input_shape)

    def call(self, X):
        return self.activation(X @ self.kernel + self.bias)

    def compute_output_shape(self, batch_input_shape):
        return tf.TensorShape(batch_input_shape.as_list()[:-1] +
          [self.units])
```



Takeaway note:
Bằng cách này, Layer tùy biến có thể nối chuỗi trong Sequential một cách trong suốt.

Gemini Notebook

Tùy chỉnh toàn bộ Mô hình (Custom Models)

Khi xây dựng kiến trúc không dạng tuần tự (VD: mạng lưới có đường chéo, Residual Neural Networks), cấu trúc Sequential trở nên cồng kềnh.

Phương pháp Model Subclassing: Cho phép ta thiết kế bằng Python hoàn toàn tự do.

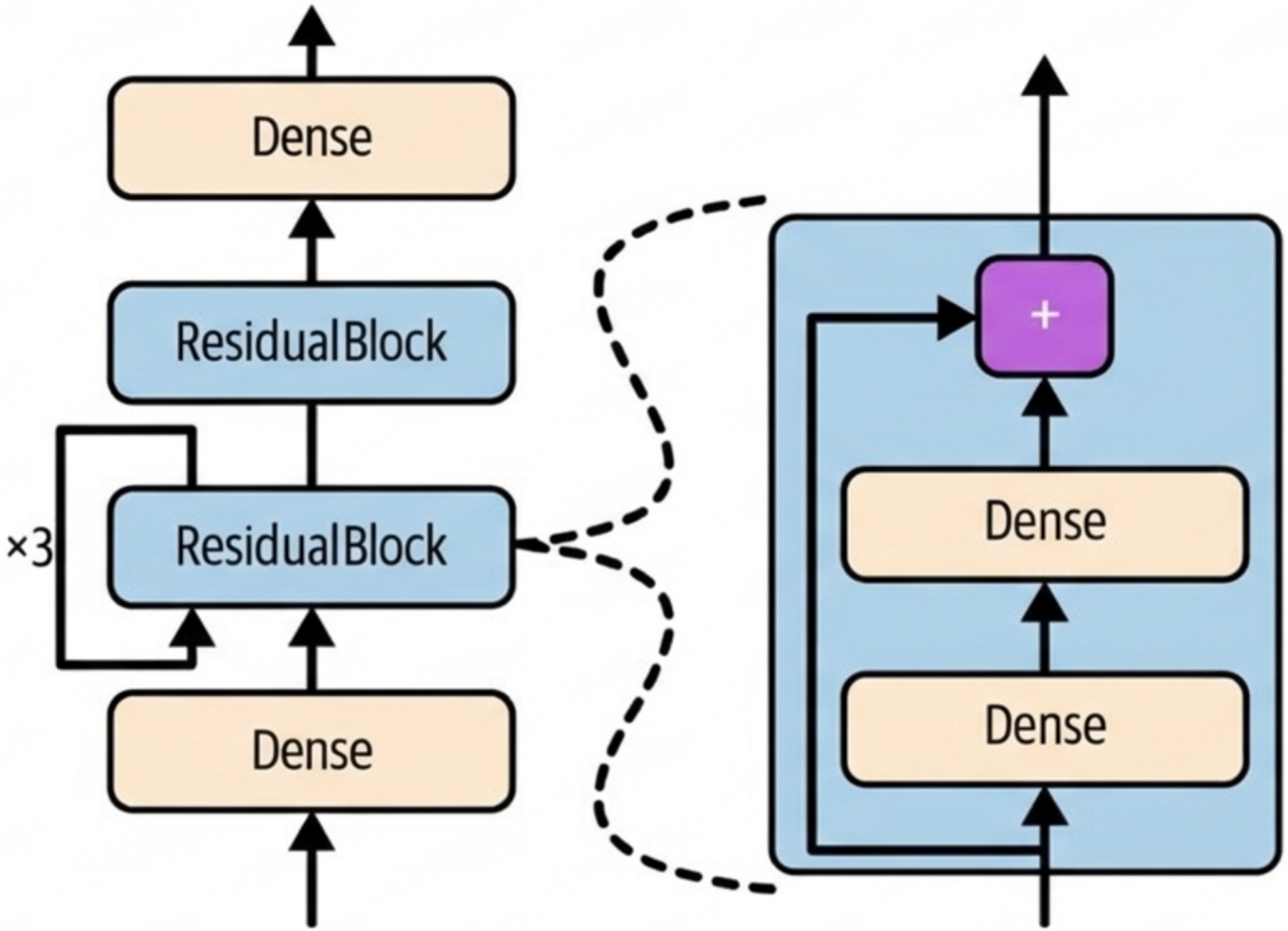
Điểm mạnh: Có thể dùng luồng điều hướng if...else, vòng lặp for để nối các module một cách linh động (dynamic).

Hướng tiếp cận: Ta sẽ trình bày mô hình mẫu sử dụng một lớp khối (Residual Block) với khả năng Skip Connection.



Gemini Notebook

Phác thảo cấu trúc một kiến trúc tùy ý với kỹ thuật Skip connection. Đường đi thẳng truyền trực tiếp Input cộng với Output của chuỗi tầng ẩn. Mục đích: Giải quyết triệt để lỗi mất gradient (Vanishing Gradient) trong Deep Learning.



Hình 12-3: *Ví dụ mô hình tùy chỉnh: một mô hình tùy ý với...*

Gemini Notebook



Mã Python Xây dựng Residual Block

Viết một Custom Layer đóng vai trò chứa chuỗi Dense và làm phép tính + kết nối.

```
class ResidualBlock(keras.layers.Layer):
    def __init__(self, n_layers , n_neurons , **kwargs):
        super().__init__(**kwargs)
        self.hidden = [keras.layers.Dense(n_neurons , activation="elu")
                       for _ in range(n_layers)]

    def call(self, inputs):
        Z = inputs
        # Vong lap for feed-forward
        for layer in self.hidden:
            Z = layer(Z)
        # Phép Cong Skip Connection
        return inputs + Z
```

Gemini Notebook

Xây dựng Custom Model sử dụng Residual Block



Kế thừa keras.Model giống kế thừa Layer, cấu trúc __init__ và call() tương tự.

```
class ResidualRegressor(keras.Model):
    def __init__(self, output_dim , **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = keras.layers.Dense(30, activation="elu")
        self.block1 = ResidualBlock(2, 30)
        self.block2 = ResidualBlock(2, 30)
        self.out = keras.layers.Dense(output_dim)

    def call(self, inputs):
        Z = self.hidden1(inputs)
        for _ in range(1 + 3): # 4 lan di qua Block 1
            Z = self.block1(Z)
        Z = self.block2(Z)     # 1 lan di qua Block 2
        return self.out(Z)
```

Gemini Notebook

Model Subclassing vs Functional API

Subclassing API

- ⚙️ Cung cấp sự linh hoạt cực lớn (có thể dùng vòng for, luồng logic động).
- ⚠️ Nhược điểm chí mạng: Keras không thể lưu đồ thị model.summary() hoặc vẽ plot_model() khi chưa truyền thử bộ dữ liệu (Model is not Built). Lỗi logic luồng xử lý bên trong Python rất khó phát hiện.



Lời khuyên (Takeaway): Hãy dùng Functional API nếu có thể, trừ phi mạng có vòng lặp động hoặc điều kiện tùy thuộc dữ liệu sinh ra (dynamic branching).

Gemini Notebook